

A LEARNING CASE STUDY

Building RL Console

How a small Tauri + Next.js + Rust desktop app was put together — and what to learn from it.

For: a developer who has touched React but not shipped a desktop app.

Stack used: Tauri 2 (Rust shell) · Next.js 16 (static export) · React 19 · TypeScript · Tailwind 4 · SQLite · Anthropic Claude API

Goal of this document: after reading it, you should be able to read every file in the repo, explain why it exists, and reproduce the same architecture for a different problem.

How to read it: in order. Each chapter builds on the previous one.

The interesting parts of a real project are the *decisions*, not the syntax. So this is structured as a story of choices: why a desktop app, why Tauri, why a static Next.js export, why two HTTP code paths, why `useRef` in some places and `useState` in others. Code samples are kept small; the prose carries the lesson.

Contents

1. The problem we're solving
2. Picking the stack
3. Mental model: how the pieces fit
4. The Rust shell — Tauri concepts
5. The frontend — Next.js as a static SPA
6. TypeScript types as contracts
7. The `lib/` layer — pure logic
8. The components — composition patterns
9. React patterns we leaned on
10. The orchestrator (`page.tsx`)
11. Persistence: three storage tiers
12. The build log — real problems we hit
13. Extending the app
14. Takeaways & further reading

01 The problem we're solving

An ops person at Rocketlane regularly needs to make API calls — sometimes one-off, sometimes hundreds of similar calls from a spreadsheet (e.g. "delete these 200 tasks"). Today they juggle Postman, hand-edited curl commands, and ad-hoc Python scripts. Each session costs them an hour of copy-pasting and praying nothing fires twice.

What we want is a small console where:

- You can describe a request in English ("create a task with start date and effort"), and an LLM drafts the curl for you.
- You can fire it once, or upload a CSV and fire it for every row.
- It auto-paginates GETs without you thinking about it.
- Everything runs locally — API keys never leave the machine.

That's the brief. Notice what's *not* there: no multi-user accounts, no cloud sync, no admin panel. Keeping the brief small is what makes the architecture small.

LESSON

Before picking a stack, write the brief in three sentences. If it grows beyond that, you're probably building two products. The "no multi-user, no cloud" constraint is what unlocks the whole desktop-app architecture below.

02 Picking the stack

Three constraints drove the choices:

1. **Local-first**. No backend to host, no auth, no servers to keep up. Each user runs their own copy.
2. **Bypass browser CORS**. Anthropic's CORS policy on the user's org blocks browser- origin requests entirely. So we need a way to make HTTP calls that *aren't* from a browser.
3. **Rich UI** with tables, modals, live-streaming logs. We want to use modern React.

Choice 1 — Tauri over Electron

Tauri wraps a system webview (no Chromium bundled) plus a Rust binary. The result is a 5–10 MB installer instead of Electron's 100+ MB, and you get Rust available for any native work you need. For us that "native work" is exactly the CORS bypass: Rust's `reqwest` isn't a browser, so Anthropic doesn't apply CORS to it.

CONCEPT — TAURI'S TWO-PROCESS MODEL

A Tauri app has two halves. The **frontend** is HTML/JS running in the OS webview (WebKit on macOS). The **backend** is a Rust binary. They talk via *commands*: Rust functions annotated with `# [tauri::command]` that the frontend calls with `invoke('command_name', args)`. Arguments are auto-serialised JSON; return values come back as a `Promise`. This is your IPC.

Choice 2 — Next.js (App Router) statically exported

We don't need a Node server. But we want familiar React DX, file-based routing, the App Router's component model, and Tailwind ergonomics. Next.js with `output: 'export'` gives us all of that and produces a folder of static HTML/JS that Tauri can bundle directly.

WHY NOT PURE VITE + REACT?

Vite would also have worked — and would arguably be simpler. We chose Next.js because the team already knew it, and because keeping the door open to add server features (if we ever need them outside Tauri) costs nothing at this scale. It's a familiarity decision, not a technical one.

Choice 3 — SQLite for persistence, localStorage for preferences

Two storage needs:

- **Structured records** — saved curls, run history. We want SQL queries. → SQLite via `tauri-plugin-sql`.
- **Single-blob preferences** — Claude API key, per-endpoint docs. Simple key/value. → `localStorage`.

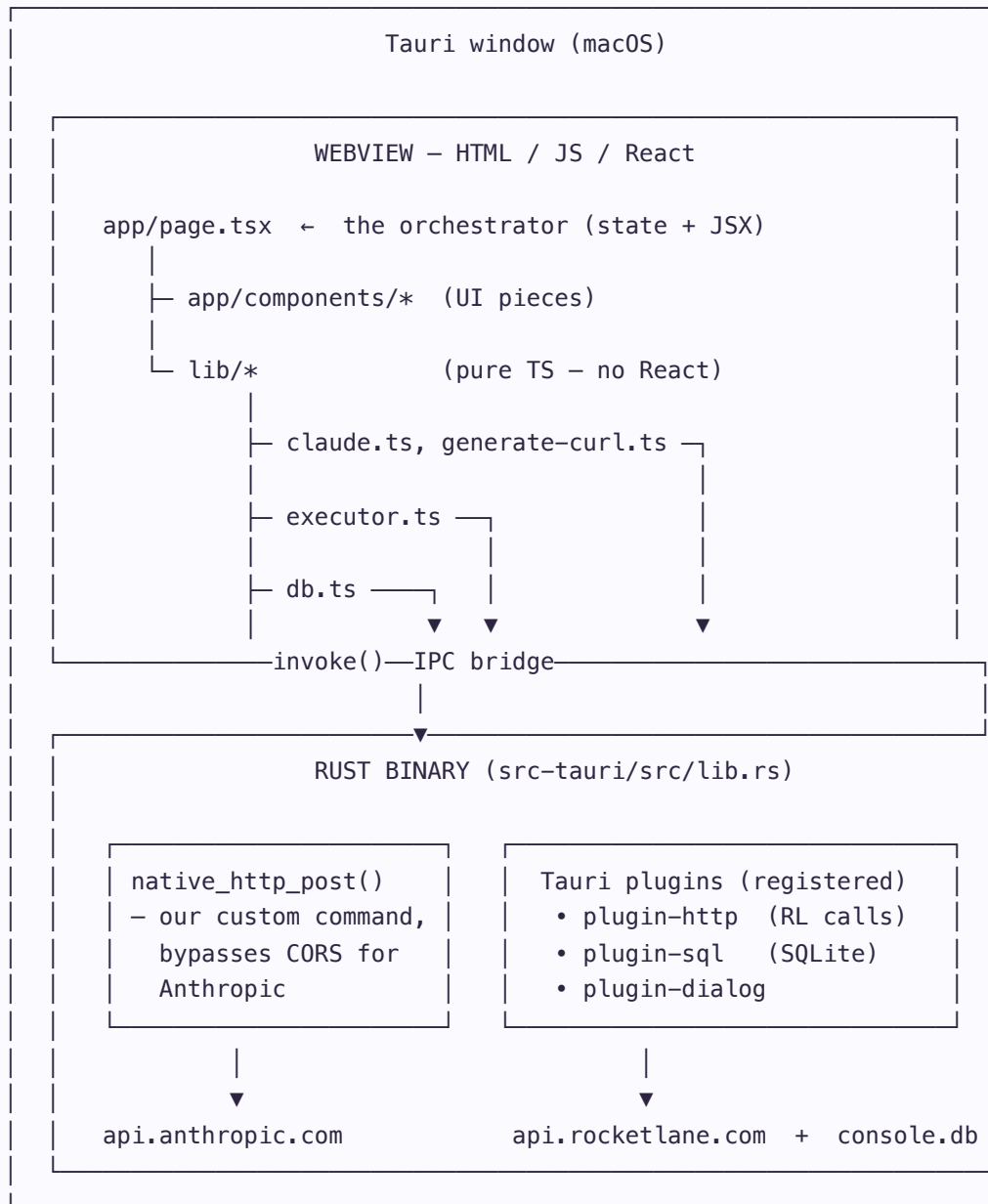
We could have used SQLite for both, or localStorage for both. Splitting them is a stylistic call that makes intent clear: *"this is structured query data"* vs. *"this is a settings blob"*.

Choice 4 — In-memory for secrets

The Rocketlane API key is **never persisted**. The user types it each session. Saving it would mean either plaintext on disk (bad) or a keychain integration (more code than the brief warrants). For an internal tool, a session-only memory store is the right trade.

03 Mental model: how the pieces fit

If you can hold this picture in your head, the rest of the doc is just zoom-ins.



Three things to internalise from the diagram:

- **The frontend never talks to the network directly.** Every HTTP call goes through *some* Tauri facility — either our custom command or the HTTP plugin.
- **The frontend never touches the disk directly** for structured data either. SQLite reads/writes go through `tauri-plugin-sql`. (Preferences *do* use the webview's `localStorage`, which is webview-native and not Tauri.)
- **There's a clean tier between React and IPC:** `lib/`. Components don't call `invoke()` themselves. They call functions in `lib/` that hide the IPC.

LESSON — KEEP IPC AT THE EDGE

When you have a Rust↔JS boundary, don't spread `invoke()` calls through your components. Wrap them in plain TS functions in a single layer (here, `lib/`). Components stay testable in a regular browser; the Rust dependency lives behind a function name.

04 The Rust shell — Tauri concepts

The whole Rust side fits on one screen. Open `src-tauri/src/lib.rs`:

```
use std::collections::HashMap;

// A Tauri command. Callable from JS as: invoke('native_http_post', {...}).
#[tauri::command]
async fn native_http_post(
    url: String,
    headers: HashMap<String, String>,
    body: String,
) -> Result<String, String> {
    let client = reqwest::Client::new();
    let mut req = client.post(&url).body(body);
    for (k, v) in &headers {
        req = req.header(k.as_str(), v.as_str());
    }
    let response = req.send().await.map_err(|e| e.to_string());
    response.text().await.map_err(|e| e.to_string())
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_dialog::init())
        .plugin(tauri_plugin_http::init())
        .plugin(tauri_plugin_sql::Builder::default().build())
        .invoke_handler(tauri::generate_handler![native_http_post])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

That's it. Three concepts to take away:

CONCEPT — `#[TAURI::COMMAND]`

Decorates a Rust async function so Tauri serialises its arguments from JSON and exposes it to the webview. Arguments are passed as an object: `invoke('name', { url, headers, body })` on the JS side becomes `(url, headers, body)` on the Rust side. Parameter *names* must match.

CONCEPT — `GENERATE_HANDLER!`

Tauri compiles a dispatch table at build time. Every command you want callable from JS must be listed in `generate_handler![...]`. Forgetting this is the #1 Tauri footgun: the command exists in Rust, compiles fine, but `invoke` from JS throws "command not found".

CONCEPT — TAURI PLUGINS

A plugin is a Rust crate that registers commands of its own. We use three: `dialog` (file open/save dialogs), `http` (HTTP from JS through Rust), and `sql` (SQLite).

`.plugin(tauri_plugin_http::init())` on the builder makes the plugin's commands callable. The plugin's JS side ships as a separate npm package — note `@tauri-apps/plugin-http` in `package.json`.

Capabilities — the security model

Open `src-tauri/capabilities/default.json`. This is where Tauri's permission system lives:

```
{
  "permissions": [
    "core:default",
    "dialog:default",
    {
      "identifier": "http:allow-fetch",
      "allow": [
        { "url": "https://api.anthropic.com/**" },
        { "url": "https://api.rocketlane.com/**" }
      ]
    },
    "sql:allow-load",
    "sql:allow-execute",
    "sql:allow-select",
    "sql:allow-close"
  ]
}
```

CONCEPT — CAPABILITIES ARE BAKED AT BUILD TIME

Tauri's HTTP plugin will refuse to fetch any URL not in the allow-list. This is a feature: even if a frontend bug or an upstream dependency tries to exfiltrate data to `evil.com`, the request will be rejected at the Rust layer. The cost is a one-line edit when you add a new host.

The configuration that connects Rust to the frontend

`src-tauri/tauri.conf.json` wires the two halves together:

```
"build": {
  "frontendDist": "../out",           // where Next.js exports to
  "devUrl": "http://localhost:3000", // where Next.js dev server runs
  "beforeDevCommand": "npm run dev", // auto-start it on `tauri dev`
  "beforeBuildCommand": "npm run build"
}
```

So `tauri dev` spawns `next dev` and points the webview at `:3000`. `tauri build` runs `next build`, which (because of `output: 'export'` in `next.config.ts`) emits static HTML/JS into `out/`,

and Tauri then bundles that `out/` directory into the `.app`.

WHY A CUSTOM `NATIVE_HTTP_POST` WHEN TAURI HAS AN HTTP PLUGIN?

The HTTP plugin still presents browser-like headers (Origin, etc.) that Anthropic's CORS gate rejects on this org. Pure `request` doesn't set those headers. So Anthropic gets Anthropic; everything else (Rocketlane) goes through the plugin via `httpFetch()`.

05 The frontend — Next.js as a static SPA

Next.js 16 is built around the App Router, server components, streaming, server actions — none of which we use, because static export disables them. What we get is essentially Next.js as a really nice SPA framework: file-based routing, opinionated defaults, automatic font/image optimisation, Tailwind out of the box.

The export switch

One line in `next.config.ts` changes everything:

```
const nextConfig: NextConfig = {
  output: 'export',
  images: { unoptimized: true },
};
```

This makes `next build` emit pure HTML/JS into `out/` instead of a Node server. The trade-offs:

- **No** server components, server actions, route handlers, middleware, ISR, dynamic segments at request time.
- **Yes** client components, all React features, file-based routing for *static* pages, Tailwind, fonts.

PITFALL

If you copy a Next.js tutorial that uses `fetch()` in a server component or a route handler under `app/api/`, it will compile but **silently fail to ship** in the export. Make sure you're writing client components only — every component file in this repo starts with `'use client'` for that reason.

The root layout and globals

`app/layout.tsx` is the only layout — it loads two Geist fonts via `next/font/google` and exposes them as CSS variables. `app/globals.css` imports Tailwind v4, defines theme tokens with `@theme inline`, and registers a handful of keyframe animations (splash logo, modal entry, spinning loader).

CONCEPT — TAILWIND V4 VS V3

Tailwind 4 dropped the JavaScript config in favour of CSS-first theming. There's no `tailwind.config.js`. You write `@theme inline { --color-foo: ... }` in your CSS, and that variable becomes a Tailwind utility (`bg-foo`, `text-foo`). Custom utilities live in `@layer utilities` blocks.

06 TypeScript types as contracts

Before any logic file, read `types/index.ts`. Five interfaces, ~50 lines. They're the lingua franca: every function signature in `lib/` and every component prop references one of these.

TYPE	LIVES WHERE	MEANS
<code>MappingTemplate</code>	Output of Claude	The HTTP request shape with placeholders, plus a mapping from each placeholder to either a CSV column or the literal <code>"session_rl_key"</code> .
<code>LogEntry</code>	Per-request result	Status, status code, message, optional raw items array (for paginated GET exports).
<code>RunSummary</code>	End-of-run counts	success / failed / skipped / dryRun.
<code>SavedCurl</code>	SQLite row	Saved curl + context + last-used timestamp.
<code>RunRecord</code>	SQLite row	One past run's metadata + outcome counts.

LESSON — TYPES ARE DOCUMENTATION

Reading these five interfaces tells you what the app actually *does*: it builds templates, runs requests, logs entries, summarises runs, and saves both curls and runs. No prose required. When you start a project, write the types first.

07 The `lib/` layer — pure logic

Seven files, no React imports. Each does one thing.

`lib/tauri-fetch.ts` — the smartest 10 lines in the repo

```
type TauriWindow = Window & { __TAURI_INTERNALS__?: unknown };

export async function httpFetch(url: string, init: RequestInit = {}): Promise<Response> {
  const w = typeof window !== 'undefined' ? (window as TauriWindow) : null;
  if (w?.__TAURI_INTERNALS__) {
    const { fetch: tauriFetch } = await import('@tauri-apps/plugin-http');
    return tauriFetch(url, init) as unknown as Response;
  }
  return fetch(url, init);
}
```

Why this matters: the same code runs in two environments. If the user opens the app via `npm run tauri dev`, `__TAURI_INTERNALS__` exists and we go through Rust. If they run `npm run dev` in a regular browser, we silently fall back to native `fetch`. Same call sites, no `if` elsewhere in the code.

CONCEPT — ENVIRONMENT DETECTION WITHOUT GLOBALS

The pattern of "if Tauri, do A; else do B" recurs in `db.ts` too. We avoid an app-level "isTauri" flag because we want each module to be self-contained and testable. Each one detects its own environment.

`lib/claude.ts` — Claude as a JSON producer

This file has one job: turn (*curl + context + sample CSV row*) into a strict JSON template. Three things make it interesting:

1. **Prompt engineering as code.** The system prompt enforces format precisely — no markdown, no commentary, exact JSON shape. We even tell Claude to map auth-looking placeholders to the literal string `"session_rl_key"`, so we can detect them later.
2. **Zod validation.** Claude is great but it's not a parser. We validate every response against a Zod schema and throw on shape mismatches. *Never trust an LLM's output structure.*
3. **It calls `native_http_post`, not the Anthropic SDK.** The SDK is installed in `package.json` but unused, because of the CORS issue.

```
const MappingSchema = z.object({
  method: z.string(),
  url: z.string(),
  headers: z.record(z.string(), z.string()),
  body: z.record(z.string(), z.unknown()).nullable(),
  mapping: z.record(z.string(), z.string()),
});

// later:
const result = MappingSchema.safeParse(parsed);
if (!result.success) throw new Error(`Invalid mapping structure: ${result.error.message}`);
```

LESSON — RUNTIME VALIDATION AT BOUNDARIES

TypeScript types vanish at runtime. Anywhere data crosses an untrusted boundary (LLM, network, user input, file upload), validate with a runtime schema. Zod is the de facto choice.

lib/generate-curl.ts — Claude as a code generator

Same Anthropic plumbing, different prompt. Returns a curl string instead of structured JSON. Notable rule in the system prompt:

"ONLY include parameters that are REQUIRED by the API, or that the user explicitly mentions. Do NOT include optional filter/query parameters just because they exist in the docs."

This rule was added after a real bug: Claude was emitting curls with 50+ optional params, which then made `extractMapping` hallucinate placeholders. *Bug fixes often live in the prompt, not the code.*

lib/rl-api-endpoints.ts — the "dumb data" file

A static catalogue: `RL_CATEGORIES` with ~60 endpoints across 12 categories, each with a key, display name, and HTTP method. Plus two functions to read/write per-endpoint docs to `localStorage`. The picker UI, the settings page, and the curl-generation flow all read from this same source.

LESSON — KEEP STATIC CONFIGURATION AS DATA, NOT CODE

Categories and endpoints are an array of objects, not `switch` statements scattered across the UI. Adding a new endpoint is a one-line append; rendering, filtering, and method badges all derive from the array.

lib/db.ts — SQLite the lazy way

Worth showing the import pattern:

```
let _db: any = null;

async function getDb(): Promise<any> {
  if (!isTauri()) return null;
  if (_db) return _db;
  try {
    const { default: Database } = await import('@tauri-apps/plugin-sql');
    _db = await Database.load('sqlite:console.db');
    return _db;
  } catch { return null; }
}
```

Three patterns here:

- **Dynamic import.** `await import('@tauri-apps/plugin-sql')` only happens inside Tauri, so the module never loads in a regular browser dev session.
- **Module-level singleton (`_db`).** Initialised once, reused everywhere.
- **Graceful no-op.** Every export starts with `const db = await getDb(); if (!db) return;` — so calling `saveRun()` in a browser does nothing instead of throwing.

`lib/executor.ts` — the actual API caller

The biggest `lib/` file (~250 lines), and the one to read carefully. It has three public exports — `buildRequest`, `validateRow`, `executeRow` — and a handful of helpers.

Placeholder substitution

```
function fill(value: string, row, rlKey, mapping) {
  return value.replace(/\{\{(\w+)\}\}/g, (match, key) => {
    const mapped = mapping[key];
    if (!mapped) return match;
    if (mapped === 'session_rl_key') return rlKey;
    return row[mapped] ?? match;
  });
}
```

`fillDeep()` is the same idea but recurses into the body's objects/arrays.

Forgiveness baked in

`cleanUrl()` strips empty/unresolved query params; `stripEmptyFields()` removes empty body fields. Together, they let users keep "kitchen-sink" curls and pass partial CSVs without the API rejecting them.

The pagination loop (GET branch)

```

while (currentUrl && pages < MAX_PAGES) {
  if (abortRef?.current) { aborted = true; break; }
  const res = await httpFetch(currentUrl, { method: 'GET', headers });
  // ... parse JSON, append items ...
  onPageFetched?.(pages, allItems.length, totalRecordCount);
  const hasMore = pg.hasMore !== false;
  const nextToken = hasMore ? (pg.nextPageToken ?? null) : null;
  if (nextToken) {
    const u = new URL(currentUrl);
    u.searchParams.set('pageToken', nextToken);
    currentUrl = u.toString();
    continue;
  }
  break;
}

```

CONCEPT — THE ABORT REF PATTERN

`abortRef` is a `{ current: boolean }` passed in from React's `useRef`. The loop checks it before each page fetch. **This works because the ref is shared by reference** — the same object the React component holds. Toggling `abortRef.current = true` from the Abort button is immediately visible to the loop. *If we'd used `useState`, the closure would have captured the initial value and never seen the update.*

CONCEPT — THE CALLBACK PATTERN FOR STREAMING PROGRESS

`onPageFetched` is an optional callback. The component passes it in to receive progress updates as they happen, without needing the executor to know about React state. This keeps `lib/` React-free while still supporting live UI updates.

`lib/export.ts` — 14 lines, one purpose

Build a CSV from an array of objects, escape quotes, trigger a download via a temporary `<a download>` click. Used by all three export buttons. No reason to factor it any further.

08 The components — composition patterns

Eight components, each in `app/components/`. Their shapes follow a few patterns worth naming.

Pattern A — Pure presentation

`HistoryView.tsx` takes `runs`, `onLoad`, `onDelete` as props. No internal state beyond what's needed for rendering. No data fetching. Fully driven from outside.

LESSON — PUSH STATE UP

When in doubt, hold state in the parent. A child that only renders props is trivially testable and reusable. The parent (`page.tsx`) becomes the single place to reason about data flow.

Pattern B — Self-contained widget with callbacks

`EndpointPicker.tsx` manages its own open/closed state, search input, click-outside detection. It exposes one prop: `onSelect(doc, method)`. The outside world doesn't care how the picker works, only what it produces.

```
useEffect(() => {
  function handler(e: MouseEvent) {
    if (ref.current && !ref.current.contains(e.target as Node)) {
      setOpen(false); setSearch('');
    }
  }
  document.addEventListener('mousedown', handler);
  return () => document.removeEventListener('mousedown', handler);
}, []);
```

CONCEPT — CLICK-OUTSIDE VIA DOCUMENT LISTENER + REF

This is the canonical pattern. Attach a `mousedown` listener to `document`; if the click target isn't inside our ref'd element, close. **Always** return a cleanup function that removes the listener — without it, every component remount leaks a listener.

Pattern C — Modal with lifecycle hooks

`MappingPreviewModal.tsx` is rendered conditionally by the parent (`{showPreview && template && <MappingPreviewModal ... />}`). It takes `onConfirm` and `onClose` callbacks; the parent decides what those mean. This is React's idiomatic way to do dialogs — no portals, no global modal manager, just conditional rendering.

Pattern D — Lifecycle-driven animation

`SplashScreen.tsx` is a mini state machine in 70 lines. It runs an entry animation, then after 2.1s flips an `exiting` state to trigger an exit animation, then at 2.65s calls `onDone()` so the parent can unmount it.

```
const [exiting, setExiting] = useState(false);
useEffect(() => {
  const t1 = setTimeout(() => setExiting(true), 2100);
  const t2 = setTimeout(onDone, 2650);
  return () => { clearTimeout(t1); clearTimeout(t2); };
}, [onDone]);
```

Note the cleanup: if the component unmounts early (e.g. user navigates away), we clear the timeouts so `onDone` doesn't fire on a dead component.

Pattern E — Prop-heavy "view" component

`LogPanel.tsx` is the most prop-rich (10 props). It consumes `logs`, `progress`, `fetchProgress`, `eta`, `summary` as inputs and exposes `onClear`, `onAbort`, `onRetry`, `onExportErrors`, `onExportAll`, `onExportItems` as actions. All of them are optional or conditionally undefined — the panel renders accordingly.

LESSON — WHEN A COMPONENT HAS 10+ PROPS, THAT'S A FEATURE

It means the parent is in control and the component is a faithful renderer. Don't reach for context or a global store just to "clean up" prop drilling at this scale. Prop drilling becomes a problem when it's deep, not when it's wide.

Pattern F — Page-as-view (not modal)

`SettingsView.tsx` is a full-page settings UI rendered when the user clicks the Settings tab. It owns its own draft state and writes through to `localStorage` on every change. The "settings as a tab, not a modal" choice gives more space for the API-docs editor.

09 React patterns we leaned on

If you only learn React from this project, these are the patterns that recur. Each one has a canonical use case in the codebase.

Pattern 1 — `useState` for what triggers re-renders

Anything that should update the UI when it changes goes in `useState`. Rule of thumb: *can the user see this? Use state.*

Pattern 2 — `useRef` for what doesn't

`useRef` gives you a mutable container that survives re-renders but doesn't trigger them. We use it three ways:

USE CASE	WHERE	WHY REF, NOT STATE
Abort flag	<code>abortRef = useRef(false)</code>	Loops in <code>executor.ts</code> see updates instantly via shared object reference; state would be captured by closure.
Run start timestamp	<code>runStartRef = useRef<number null>(null)</code>	Used inside <code>runRows</code> to compute ETA — doesn't need to re-render.
DOM element handles	<code>fileInputRef</code> , <code>bottomRef</code>	Programmatic <code>.click()</code> , <code>.scrollIntoView()</code> .
Toast timer handle	<code>toastTimer = useRef<Timeout null>(null)</code>	To <code>clearTimeout</code> the previous timer when a new toast arrives.

CONCEPT — CLOSURE CAPTURE & WHY `USEREF` FOR ABORT

When `runRows` starts, it captures whatever variables are in scope. If `abort` was a `useState` boolean, the loop's view of it would be frozen at whatever value it had when the function started. A ref isn't captured by value — it's captured by identity, and `ref.current` is read fresh each iteration.

Pattern 3 — `useMemo` for derived values

Two examples in `page.tsx`:

```
const placeholders = useMemo(() => parsePlaceholders(curlCmd), [curlCmd]);
const detectedMethod = useMemo(() => {
  const m = curlCmd.match(/-X\s+(\w+)/i);
  if (m?.[1]) return m[1].toUpperCase();
  if (/--G\b|--get\b/.test(curlCmd)) return 'GET';
  if (/--d\s|--data(?:-urlencode)/.test(curlCmd)) return 'POST';
  return 'GET';
}, [curlCmd]);
```

Both are pure functions of `curlCmd`. We `useMemo` them so they only re-run when the curl text actually changes, not on every keystroke in unrelated fields. **Don't overuse `useMemo`** — for cheap calculations, the bookkeeping costs more than the work. We use it here because `parsePlaceholders` drives a `useEffect`, and we want stable identity.

Pattern 4 — `useEffect` for syncing external systems

Three uses in this app:

- **Mount-only setup** — `useEffect(() => { initDb(); ... }, [])`. Empty dependency array = run once after the first render.
- **Sync derived state** — when `placeholders` changes, sync the `singleValues` dict to have the same keys. Effect runs when its dependency changes.
- **Subscribe / unsubscribe** — click-outside listener in `EndpointPicker`. Always return a cleanup function.

PITFALL — PUTTING DERIVED STATE IN `USEEFFECT` + `USESTATE`

If you find yourself writing `useEffect(() => setX(compute(y)), [y])`, that's a `useMemo` in disguise — and worse, because it triggers an extra render. Use effects for *side effects* (DOM, network, subscriptions), not for computing values.

Pattern 5 — Controlled inputs everywhere

Every text input in the app has both `value` and `onChange` wired to React state:

```
<input
  type="password"
  value={rlKey}
  onChange={(e) => setRlKey(e.target.value)}
/>
```

This makes the input a function of state. We can clear it (`setRlKey('')`), prefill it (e.g. from a saved curl), or validate it on every keystroke. Uncontrolled inputs are an optimisation we don't need.

Pattern 6 — Conditional rendering as the routing primitive

This app has no React Router. Three "pages" — Operation, History, Settings — are switched by `activeView` state and a ternary in JSX:

```
{activeView === 'settings' ? (  
  <SettingsView ... />  
) : activeView === 'history' ? (  
  <HistoryView ... />  
) : (  
  <OperationView ... /> // inlined in this app, but conceptually one  
)}
```

For three views, this is fine. If you grow to ten, switch to a router.

Pattern 7 — Lifting state to a single owner

`page.tsx` holds *almost all* state. Components consume it via props. The exceptions are local UI state (an open/closed dropdown, a tab selection in a modal) which doesn't need to leave the component.

Lifting state up is what lets us derive `canRun`, `canPreview`, etc. from the same source of truth.

10 The orchestrator (`page.tsx`)

~990 lines. The size is intentional: by keeping all the wiring in one file, the data flow is trivial to trace. There's no "how does this state get to that component" hunt — it's all here.

The shape of the file

1. **Top utilities** (lines ~20-69) — pure functions: `parsePlaceholders` , `logToRow` , `curlToName` , `parseCurlDirect` .
2. **State declarations** (lines ~72-145) — grouped by concern with comments.
3. **Effects** — mount-time setup, placeholder sync.
4. **Handlers** — one per user action: `handleSaveCurl` , `handleLoadCurl` , `handleGenerateCurl` , `handlePreviewMapping` , `handleRun` , `handleRetryFailed` , three exports.
5. **The bulk runner**: `runRows()` — the loop that fires per-row requests, accumulates summary, computes ETA after each row, sleeps the configured delay.
6. **Derived flags** — `canPreview` , `canRun` , `canRetry` , `hasGetItems` .
7. **Render** — the JSX tree. Header → main (view switcher) → toast → modals → splash overlay.

Reading order to understand it end-to-end

1. State declarations first — these are the variables.
2. `handleRun` next — this is the main verb. It branches on GET vs single vs bulk.
3. `runRows` — the bulk loop, where ETA is computed.
4. `handleGenerateCurl` , `handlePreviewMapping` — the Claude-touching handlers.
5. JSX last — by now you'll recognise every variable being passed around.

LESSON — SOMETIMES ONE BIG FILE IS RIGHT

The temptation when a file hits 500 lines is to split it. Resist for a beat. `page.tsx` is big because it owns the state graph. Splitting it would mean either inventing a context (more indirection) or duplicating state (worse). The file is long but the *complexity* is contained — every variable is in scope, every handler is two scrolls away.

The ETA computation in detail

This was added late in the project. The full mechanism:

```
// Inside runRows(), after each row completes:
const completed = i + 1;
const remaining = rows.length - completed;
if (remaining > 0 && runStartRef.current !== null) {
  const elapsed = Date.now() - runStartRef.current;
  const avgMs = elapsed / completed;
  const remainingMs = remaining * avgMs;
  setEta({ remainingMs, doneAt: new Date(Date.now() + remainingMs) });
}
```

Three things to notice:

- **The start time is in a ref** (`runStartRef`), not in state — we don't need a re-render when it's set.
- **The ETA is in state** (`setEta`) — we want the LogPanel to re-render when it changes.
- **Wall-clock time is computed once** as a real `Date` , not as a relative duration to format later. The component receives a `Date` object and formats it with `toLocaleTimeString()` .

11 Persistence: three storage tiers

This pattern recurs in any local-first app, so it's worth naming explicitly.

TIER	WHAT LIVES THERE	WHY THIS TIER
In-memory (React state)	RL API key, current curl, current CSV, logs	Either secret, or transient, or both. Disappears on quit. Intentional.
localStorage	Claude API key, per-endpoint API docs	Single-blob preferences. No structure needed. Convenient.
SQLite (Tauri plugin)	Saved curls, run history	Structured, queryable, indexable. We want to sort by <code>last_used_at</code> , filter by date, etc.

LESSON — PICK STORAGE BY THE ACCESS PATTERN

Asking "where should this go?" usually has an obvious answer once you ask "what queries will I run on it?" If you'll only ever `JSON.parse(blob)`, `localStorage`. If you'll `SELECT WHERE`, `SQLite`. If it's a secret or transient, memory.

Backup & restore

The Settings page exports a JSON file with `{ claudeKey, docs }` and lets the user restore it later. This was added after a real incident: switching from `npm run tauri dev` to a built `.dmg` uses different effective storage paths in some Tauri configurations, and the user lost everything. Now they can guard against that.

12 The build log — real problems we hit

Documenting the hard parts is the most useful thing you can do for future-you. Each of these shipped in the codebase.

Problem 1 — Anthropic CORS

First attempt: call `api.anthropic.com` from the webview using the Anthropic SDK. Response: `"CORS requests are not allowed for this Organization because of custom retention settings"`. The org admin couldn't change the policy.

Second attempt: pass `anthropic-dangerous-direct-browser-access: true` per the SDK's docs. Same error — the policy overrides the header.

Third attempt: route through Tauri's HTTP plugin. Still failed — the plugin sets browser-style headers that triggered the same gate.

Fourth attempt (shipped): a custom Rust command (`native_http_post`) using `request`. Pure Rust, no browser-style headers. Anthropic accepts it. Cost: 15 lines of Rust.

LESSON

When a vendor's policy is in your way, climb a layer. We couldn't change Anthropic's policy or the SDK, so we changed the layer the request originated from.

Problem 2 — GET pagination running forever

Initial loop: `while (nextPageToken) { fetch; }`. Some Rocketlane endpoints return a stale token alongside `hasMore: false`. We were spinning.

Fix: `const hasMore = pg.hasMore !== false; const nextToken = hasMore ? pg.nextPageToken : null;`. The `!== false` phrasing matters because `hasMore` is sometimes absent (treat as more) and sometimes `false` (stop).

Problem 3 — Abort that didn't abort

Original Abort button only checked `abortRef.current` in the row-level loop, not in the page-level loop inside `executeRow`. A long GET would keep paginating.

Fix: pass `abortRef` down into `executeRow` and check it before each page fetch. Aborted runs return their partial results as `'skipped'`.

Problem 4 — Logs duplicating across runs

First version: `setLogs((prev) => [...prev, ...newEntries])` in the run loop. Each new run appended to the previous run's logs. Off-screen, the panel would scroll forever.

Fix: `setLogs([])` at the start of every run, then build up. Trivial in hindsight, but easy to write the wrong way the first time.

Problem 5 — Claude generating curls with 50 query params

The first version of `generate-curl.ts` just said "generate a curl from this description and these docs". Claude faithfully included every optional filter parameter the docs mentioned. The downstream `extractMapping` then hallucinated CSV columns to match them.

Fix: hard-coded constraint in the system prompt — "*ONLY include parameters that are REQUIRED or that the user explicitly mentions*". Plus a special case for GET list endpoints to only include `pageSize` and `pageToken`.

LESSON — PROMPTS ARE PART OF YOUR CODEBASE

When an LLM-driven feature misbehaves, the fix is usually in the prompt, not in TypeScript. Treat prompts with the same rigour as code: version them, test them, document why each line is there.

Problem 6 — "Settings disappeared after .dmg reinstall"

localStorage was scoped to the dev origin (`localhost:3000`) when running `tauri dev`, but to the bundled app's webview origin in production. Switching between the two looked like data loss.

Workaround (shipped): Settings → Export/Import as JSON. Long-term fix would be to migrate prefs into SQLite too — same identifier, same path on disk regardless of dev vs prod.

13 Extending the app

Concrete recipes. Each maps to one or two file edits.

Add a new Rocketlane endpoint

One file: `lib/rl-api-endpoints.ts`. Append to the appropriate category. The picker, settings sidebar, and badges pick it up.

Allow a new external HTTP host

One file: `src-tauri/capabilities/default.json`. Add a URL pattern. Rebuild.

Tweak Claude's curl-generation behaviour

One file: `lib/generate-curl.ts`. Edit the `SYSTEM` string. Smoke-test against *Get all tasks* (the worst-case for over-eager parameters).

Add a column to run history

Three files:

1. `types/index.ts` — add the field to `RunRecord`.
2. `lib/db.ts` — update `CREATE TABLE` in `initDb` (and write a migration if anyone has a real DB already).
3. `app/components/HistoryView.tsx` — new `<th>` + `<td>`.

Add a new top-level view

1. Create `app/components/StatsView.tsx`.
2. In `app/page.tsx`: add `'stats'` to the `activeView` union; add a button to the nav array; add a render branch.

Change the Claude model

Two files: `lib/claude.ts` and `lib/generate-curl.ts`. Both have `const MODEL` at the top. (Or refactor into a single shared constant — it's currently duplicated.)

14 Takeaways & further reading

Five things to take away

1. **Match the architecture to the brief.** Local-first, single-user, no backend — this is what unlocks Tauri + static export. If the brief grows, the architecture has to grow with it.
2. **Layer your code so each layer has one reason to change.** Rust shell, lib/ pure logic, components, page orchestrator. When a bug appears, you know which layer to look in.
3. **Validate at boundaries.** Zod for LLM output, capabilities for HTTP destinations, runtime detection for the Tauri/browser split. TypeScript types are not a substitute.
4. **Pick state primitives by access pattern.** `useState` when the UI must react. `useRef` when it must not. `useMemo` for derived values. `useEffect` only for genuine side effects.
5. **Document the why, not the what.** Code shows what. Prose, comments, and prompt rules show why. The "build log" chapter above is more valuable than any function comment.

Things this project does *not* do (and could)

- **Tests.** No unit tests. The `lib/` layer was built to be testable (no React imports), but no Jest/Vitest setup exists. Easy first contribution.
- **Migrations.** `db.ts` uses `CREATE TABLE IF NOT EXISTS` only — no schema versioning. Fine for a single user; would need a migration system for a real rollout.
- **Code splitting.** The whole app is one route, so this matters less, but the bundled JS is bigger than it needs to be. Lazy-loading the modals would shave kilobytes.
- **Keychain integration.** The Claude key is in `localStorage`, not the macOS keychain. Acceptable for an internal tool, not for distribution.

Suggested reading, in order

1. **Tauri 2 docs:** <https://tauri.app/start/> — particularly the "Commands" and "Capabilities" sections.
2. **React 19 release notes** — focus on the new `use()` hook and form actions (which we don't use here, but you'll meet them).
3. **Next.js 16 App Router** + the `output: 'export'` guide. Everything else (server components, server actions) doesn't apply when you're statically exporting.
4. **Anthropic API reference** — particularly the messages endpoint shape and how system prompts work. The format of our request body in `claude.ts` mirrors it.
5. **Zod docs** — `z.object`, `safeParse`, refinements. Useful far beyond this project.

End of case study. The best next step is to clone the repo, run `npm run tauri dev`, and trace one full flow with the dev tools open. Pick the bulk-run flow — it touches every layer.